AI Smart Navigation Assistant for Visually Impaired People Using Raspberry Pi

Smart Cane Environmental Awareness System

Final Presentation · IoT Term Project

Igor Xavier · **Fang Jialuo**

Outline

1. Problem
2. Solution
3. AI Technologies
4. Results
5. Future Work

Motivation & Problem

Visually impaired users face daily challenges navigating indoor spaces safely and independently — especially unfamiliar environments.

Existing solutions typically fall into two categories:

Full SLAM systems
Accurate, but expensive: LiDAR, IMU, heavy compute. Out of reach for a low-cost student project.

Simple ultrasonic canes
Cheap, but "dumb": beep when close, no semantic information — no idea *what* or *where*.

> *"Our goal: meaningful semantic awareness, on a single Raspberry Pi, fully offline, at low cost."*

Project Idea

A smart cane that answers three questions in real time:

Question	How
What is in front of me?	ToF depth detection + YOLOv8-nano classification
Where am I?	ORB visual place recognition
How do I get to my destination?	Voice input + turn-by-turn audio guidance

> *"Where do you want to go?" → "Kitchen"*
> *"Chair ahead, 0.8 metres." → "Turn left." → "You have arrived."*

Scope Decision — From SLAM to Awareness

Original vision

Full Visual SLAM (ORB-SLAM3): build a 3D map in real time, plan routes autonomously.

➡️ Needs IMU, heavy compute, careful calibration — not achievable reliably in the available time.

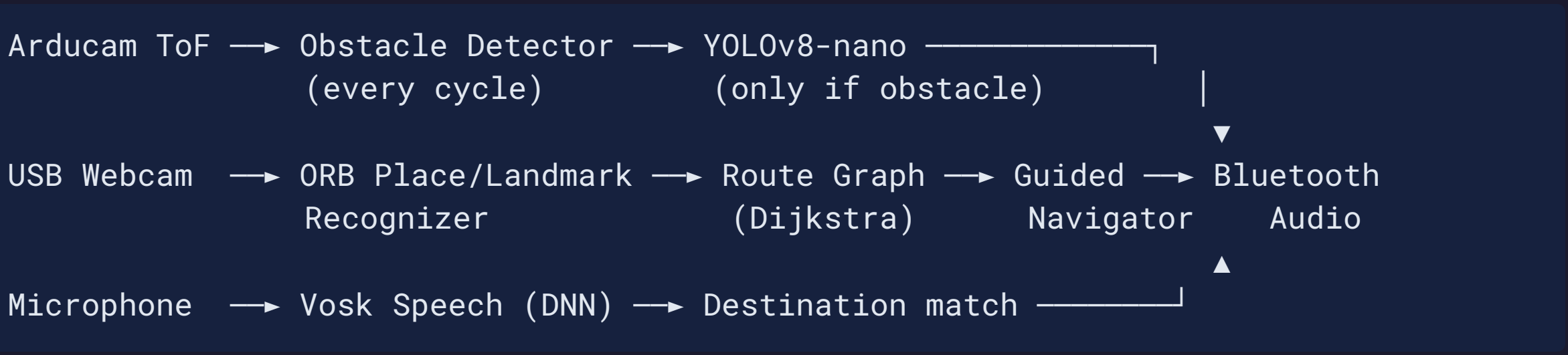
Revised, shipped scope

Two achievable modes:

- 🔵 **Awareness** — passive obstacle + location alerts
- 🟢 **Guided navigation** — voice destination + reactive turn-by-turn

SLAM preserved as documented future work.

System Architecture



Stage	Technique	Runs
Obstacle detection	Depth threshold	Every cycle (2 Hz)
Obstacle classification	YOLOv8-nano (DL)	Only when flagged
Place/landmark recognition	ORB (classical CV)	Every 3rd cycle
Route planning	Dijkstra (RouteGraph)	Once, at start
Speech-to-text	Vosk (DL)	Once, at start

Mode 1 — Awareness

Passive loop: no destination, just continuous environmental feedback.

```
def _check_obstacles(self):
    alert = self._obstacle_detector.check(tof_frame)
    if alert:
        message = self._obstacle_detector.alert_message(alert)
        if self._object_classifier.is_available:
            detection = self._object_classifier.classify(webcam_frame)
            if detection:
                message = f"{detection.label} ahead, {alert.min_depth:.1f}m."
        self._audio.alert(message)
```

Output: *"You are in the office."* · *"Chair ahead, 0.8 metres."*

Mode 2 — Guided Navigation

At the start, the system localizes the user and **plans a route** over previously trained connections between locations (Dijkstra). Then, every cycle:

```
Planned step says "forward" → ToF center zone clear? → "Go straight" : "Path blocked"
Planned step says "turn"    → announce the turn directly
No route available          → fall back to reactive wall-following (clearer side wins)
```

The depth sensor is always the final check — a planned step is only followed if it's actually safe right now. Combined with **arrival detection** and **landmark announcements** (trained doors) along the way.

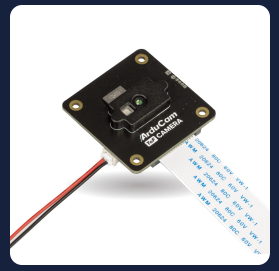
Limitations

- Routes only exist between locations explicitly connected during training — no continuous map
- Without a trained route, navigation falls back to reactive obstacle avoidance
- Obstacle detection always works, regardless of position
- Room and landmark recognition may fail under very different lighting or viewpoints

Hardware — Assembled Prototype

Hardware Components

Raspberry Pi 4
Central compute



Arducam ToF B0410
Depth, up to 4m

Logitech C270
Recognition + YOLO

BT Headphones
Audio I/O

Total hardware cost: **~€80** — no GPU, no cloud, no internet required.

Software Architecture

Module	Responsibility
`sensors/`	Webcam + ToF abstraction, auto V4L2 device detection
`mapping/`	Waypoint store (SQLite + `.npy`); `location` vs `landmark`
`localization/`	ORB place/landmark recognition
`obstacle/`	ToF depth threshold
`perception/`	**YOLOv8-nano** obstacle classification (DL)
`speech/`	**Vosk** offline speech-to-text (DL)
`navigation/`	Reactive wall-following guidance
`audio/`	espeak-ng TTS + alert tones

33 automated software tests across the codebase.

AI / Deep Learning Components

Technique	Type	Training data	Used for
ORB + BFMatcher	Classical CV	Reference images, no labels	Place / landmark recognition
YOLOv8-nano	**CNN (DL)**, pretrained	None (COCO weights)	Classify flagged obstacles
Vosk	**DNN-HMM (DL)**, pretrained	None (public model)	Offline speech-to-text

Why gated, not continuous? Running a CNN every cycle is too slow for real-time RPi4 CPU. YOLO only runs *after* the cheap ToF threshold already flagged something close — classifying what's already known to be near.

Why not YOLO for doors? "Door" isn't a COCO class — ORB landmarks fill that gap with zero labeled data.

Dataset Sources & Accuracy

Technique	Dataset source	Published accuracy
ORB	**None** — self-collected reference images from the user's own space	N/A (not a learned model)
YOLOv8-nano	**COCO** — 80 classes, ~330K images, ~1.5M instances	$mAP_{50–95}$ = 37.3 (COCO val2017, Ultralytics)
Vosk	Public English speech corpora (Alphacephei)	WER 9.85% (LibriSpeech test-clean)

Validation: beyond the published metrics, we tested our own recognition pipeline directly — automated tests for every decision module, plus a controlled robustness experiment on the exact production matching code (next slide).

ORB Robustness — Real Test Results

Tested the exact production matching code against synthetic perturbations of a real reference image:

- **Rotation** — little to no effect on recognition
- **Motion blur** — degrades recognition once blur becomes significant
- **Poor lighting** (darker) — degrades recognition faster than blur

> *“Lighting and motion matter more than camera angle — the key factor for reliable recognition in practice.”*

Code — Gated DL Inference

perception/object_detector.py

```
def classify(self, bgr_frame: np.ndarray) -> Optional[DetectionResult]:
    if not self.is_available:
        return None

    results = self._model.predict(bgr_frame, verbose=False)[0]
    if results.boxes is None or len(results.boxes) == 0:
        return None

    best_idx = int(results.boxes.conf.argmax())
    confidence = float(results.boxes.conf[best_idx])
    if confidence < self._confidence_threshold:
        return None

    class_id = int(results.boxes.cls[best_idx])
    return DetectionResult(label=self._model.names[class_id], confidence=confidence)
```

Code — Planned Route With a Safety Check

navigation/guided_navigator.py

```
def _follow_planned_route(self, left, center, right):
    edge = self._route[self._route_index]
    hint = edge.direction_hint
    if hint == "forward":
        if center <= 0 or center > self._clear_distance:
            return GuidanceResult(NavCommand.STRAIGHT, edge.audio_instruction)
        return GuidanceResult(NavCommand.STOP, "Path blocked.")
    elif hint == "turn_left":
        return GuidanceResult(NavCommand.TURN_LEFT, edge.audio_instruction)
    elif hint == "turn_right":
        return GuidanceResult(NavCommand.TURN_RIGHT, edge.audio_instruction)
    else:
        return self._wall_following(left, center, right)
```

No route available? Falls back to the same reactive wall-following — never a hard failure.

Results & Current Status

Phase	Status
Sensor integration	✅ Complete — validated on real RPi hardware
Location training	✅ Complete — 1000–5000 ORB descriptors per location
Awareness loop	✅ Complete — running end-to-end
Guided navigation	✅ Implemented & tested — voice, navigation, landmarks, arrival
YOLO obstacle classification	✅ Implemented & tested — gated DL inference

All core decision-making logic is implemented and tested — 33/33 automated tests passing.

Contributions

- A working, **fully offline** assistive prototype combining classical CV and **two deep learning models** on a single Raspberry Pi 4
- A deliberate hybrid architecture — **ORB + YOLO + Vosk + Dijkstra route planning** — each technique used where it performs best, with the depth sensor as a constant safety check
- Validated through **33 automated tests** and a real robustness experiment on the production recognition code

Challenges & Solutions

1. Full SLAM was infeasible — no IMU, no odometry, CPU-only Pi. We treated this as a scope decision, not a failure: redesigned around reactive wall-following + ORB recognition, kept SLAM as documented future work.

2. Training data evolved through 3 iterations — single frame (unreliable) → 5-frame burst (still too similar) → 2-minute continuous capture, ~40 frames, up to 5000 descriptors (final).

3. Cross-platform threading bug, found *this week* — building a desktop demo without the Pi revealed `pyttsx3` hangs when one engine is shared across threads on Windows (COM apartment threading). Fixed with a fresh engine per call.

4. DL inference too slow for continuous use — gated YOLO behind the cheap ToF threshold, paying CNN cost only when an obstacle is already confirmed.

Future Work

Deliberately deferred from the original vision:

- 🗺️ **Visual SLAM** — real-time 3D mapping (ORB-SLAM3), enabling navigation to unseen destinations
- 🧭 **Graph-based route planning** — `WaypointEdge` data model already exists; add Dijkstra across multiple legs
- 📐 **IMU integration** — dead-reckoning between locations
- 🎯 **Fine-tuned object detection** — custom classes (doors, stairs, curbs) instead of relying on COCO + ORB
- 🏠 **Multi-environment support** — automatic environment selection

Thank You

Smart Cane Environmental Awareness System

AI Smart Navigation Assistant for Visually Impaired People

Igor Xavier · **Fang Jialuo**

`github.com/w1ggor/smart-nav-cane`

Questions?